OPEN DEVELOPER GUIDE — MLX_CLXV V2.2

MiniLibX

The Complete Python Guide for Building Graphical Applications

VERSION	PLATFORM	LANGUAGE	WRITTEN BY
2.2 (CLXV)	Linux / XCB / Vulkan	Python 3 (ctypes)	Bruno Gomes

CONTENTS

01 What is MLX?
02 Architecture
03 Getting Started
04 Init & Teardown
05 Windows
06 Drawing Directly
07 Images
08 Loading Files
09 Event Loop
10 Keyboard Hooks
11 Mouse Hooks
12 Expose Hook
13 Loop Hook
14 Generic Hook
15 Mouse Utilities
16 Keyboard Utilities
17 Screen Info
18 Sync & Flush
19 Color System
20 Practical Recipes
21 Limitations
22 Quick Reference

01 What is MLX?

MinilibX (MLX) is a minimal graphical library originally developed by Olivier Crouzet. Its purpose is simple: let you open a window, draw pixels in it, load images, and react to keyboard and mouse events — without needing to know Vulkan, X11, or any other low-level graphics system.

It is intentionally kept small and low-level. There are no built-in widgets, no UI toolkit, no audio — just the raw building blocks. Everything beyond those basics is yours to design and build, which makes it a fantastic foundation for games, visualisations, simulations, tools, and any creative project that needs a graphical window.

This guide documents every function, every event, and every technique available in the Python version of MLX (`mlx_CLXV v2.2`), written so that any developer — beginner or experienced — can pick it up and start building immediately.

WHAT IT PROVIDES

- Open one or more windows on screen at any size
- Draw pixels directly or via fast image buffers
- Render text anywhere with a built-in bitmap font
- Load XPM and PNG image files as drawable sprites or backgrounds
- React to keyboard presses and releases with full keycode access
- React to mouse clicks, scroll wheel, and button press/release
- Control the mouse cursor — hide it, move it, or track its position
- Detect mouse enter/leave events on any window
- Manage multiple independent windows simultaneously
- Synchronise rendering to prevent tearing or race conditions

WHAT YOU BUILD ON TOP OF IT

- 2D games — platformers, puzzle games, maze games, shooters
- Simulations — fractals, cellular automata, physics, pathfinding visualisers
- Creative tools — pixel editors, drawing apps, generative art
- UI — clickable menus, hover effects, animated transitions, HUDs
- Data visualisation — real-time graphs, heatmaps, algorithm step-throughs

WHAT IT DOES NOT PROVIDE (YOU MUST BUILD OR SOURCE EXTERNALLY)

- Audio, networking, file I/O
- UI widgets (buttons, sliders, checkboxes) — implement with images + hit-testing
- Custom fonts or variable text sizes — only one fixed bitmap font is included

- 3D rendering, shaders, or direct GPU programming

02 Architecture

```
Your Python Code
      |
      ▼
  Mlx class (mlx.py)        ← Python ctypes wrapper
      |
      ▼
  libmlx.so                 ← compiled C library
      |
      ├── XCB backend       ← window management + events (keyboard, mouse)
      └── Vulkan backend    ← GPU drawing (pixels, images, text)
```

The **main loop** (`mlx_loop`) is an infinite event loop inside the C library. It waits for events, calls your registered Python callbacks when they occur, and calls your loop hook on every idle frame. Your program only runs through callbacks once the loop starts.

03 Getting Started

Install the MLX Python package into your virtual environment:

```
# From the mlx_CLXV directory
pip install mlx-2.2-py3-none-any.whl
```

Minimum working program — a window that closes on ESC or the X button:

```
from mlx import Mlx
import os

def on_key(key, param):
    if key == 65307:  # ESC
        os._exit(0)

mlx = Mlx()
mlx_ptr = mlx.mlx_init()
win_ptr = mlx.mlx_new_window(mlx_ptr, 800, 600, "My Window")

mlx.mlx_key_hook(win_ptr, on_key, None)
mlx.mlx_hook(win_ptr, 33, 0, lambda p: os._exit(0), None)  # close button

mlx.mlx_loop(mlx_ptr)
```

04 Initialization & Teardown

`mlx_init()`

```
mlx_ptr = mlx.mlx_init()
```

Must be called **first**, before anything else. Creates the connection to the display system (XCB + Vulkan).

> **Returns:** connection identifier — pass this to most other functions. Returns `None` on failure.

`mlx_release(mlx_ptr)`

```
mlx.mlx_release(mlx_ptr)
```

Disconnects from the display and frees all internal resources. Call at the end of your program after the loop exits.

> **Returns:** `0` on success.

05 Windows

`mlx_new_window(mlx_ptr, width, height, title)`

```
win_ptr = mlx.mlx_new_window(mlx_ptr, 1600, 900, "A-MAZE-ING")
```

Creates a new window on screen. MLX can manage **multiple windows simultaneously** — each call creates a separate one.

`width, height`	Window size in pixels.
`title`	Text shown in the window's title bar. Auto-encoded to UTF-8.

Returns: window identifier (pass to drawing and hook functions). `None` on failure.

ℹ Origin `(0, 0)` is the **top-left corner**. X increases right, Y increases downward.

`mlx_clear_window(mlx_ptr, win_ptr)`

```
mlx.mlx_clear_window(mlx_ptr, win_ptr)
```

Fills the entire window with **black**. Use this to erase everything before drawing a new frame.

`mlx_destroy_window(mlx_ptr, win_ptr)`

```
mlx.mlx_destroy_window(mlx_ptr, win_ptr)
```

Closes and destroys a window. Useful when managing multiple windows and closing just one. The pointer becomes invalid after this call.

06 Drawing Directly

`mlx_pixel_put(mlx_ptr, win_ptr, x, y, color)`

```
mlx.mlx_pixel_put(mlx_ptr, win_ptr, 100, 200, 0xFFFFFFFF)
```

Draws a **single pixel** at position `(x, y)` with the given color. Most basic drawing primitive.

⚠ `mlx_pixel_put` is **slow** — each pixel is a separate GPU command. For bulk drawing use image buffers instead (see Section 07).

`mlx_string_put(mlx_ptr, win_ptr, x, y, color, string)`

```
mlx.mlx_string_put(mlx_ptr, win_ptr, 100, 50, 0xFFFFFFFF, "Hello World")
```

Renders a text string at `(x, y)` using MLX's built-in **fixed bitmap font**. Supports ASCII 32–127 only.

`x, y`	Top-left corner of the first character.
`color`	Text color. Alpha channel is forced to 255 (opaque) — semi-transparent text is not supported.
`string`	Text to display. Auto-encoded to UTF-8.

ℹ Character width ≈ 7–8 px. Character height ≈ 13 px. Font size and face cannot be changed.

07 Images

Images are the **recommended way to draw**. You build the image in memory, then send it to the GPU in one call — much faster than pixel-by-pixel.

`mlx_new_image(mlx_ptr, width, height)`

```
img_ptr = mlx.mlx_new_image(mlx_ptr, 200, 200)
```

Creates a blank image in memory of the given size.

Returns: image identifier, or `None` on failure.

`mlx_get_data_addr(img_ptr) → tuple`

```
data, bpp, size_line, fmt = mlx.mlx_get_data_addr(img_ptr)
```

Returns a **writable memoryview** of the image's raw pixel buffer plus metadata. This is the key function for drawing into images.

`data`	Writable `memoryview` (cast to `'B'`) — the raw pixel bytes.
`bpp`	Bits per pixel — typically `32`.
`size_line`	Bytes per row. Use this to move between rows: `offset = y * size_line + x * 4`
`fmt`	`0` = B8G8R8A8 byte order | `1` = A8R8G8B8 byte order

Writing pixels:

```
# Put a white pixel at (10, 20)
offset = 20 * size_line + 10 * 4
data[offset:offset + 4] = bytes([0xFF, 0xFF, 0xFF, 0xFF])

# Fill entire image red
for offset in range(0, size_line * height, 4):
    data[offset:offset + 4] = bytes([0x00, 0x00, 0xFF, 0xFF])  # BGRA red
```

`mlx_put_image_to_window(mlx_ptr, win_ptr, img_ptr, x, y)`

```
mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, img_ptr, 0, 0)
```

Renders an image onto the window at `(x, y)`. Images drawn later appear on top. `x, y` can be negative for partially off-screen images.

`mlx_destroy_image(mlx_ptr, img_ptr)`

```
mlx.mlx_destroy_image(mlx_ptr, img_ptr)
```

Frees image memory. Always destroy images you no longer need to avoid memory leaks.

08 Loading Image Files

`mlx_xpm_file_to_image(mlx_ptr, filename) → tuple`

```
img_ptr, width, height = mlx.mlx_xpm_file_to_image(mlx_ptr, "assets/wall.xpm")
```

Loads an **XPM file** into an image. **XPM is the recommended format** for this MLX version — most reliable loader. Supports transparency.

✓ Convert any PNG/JPEG to XPM with ImageMagick: `magick input.png output.xpm`

`mlx_png_file_to_image(mlx_ptr, filename) → tuple`

```
img_ptr, width, height = mlx.mlx_png_file_to_image(mlx_ptr, "assets/wall.png")
```

Loads a **PNG file** into an image.

⚠ The PNG loader is incomplete and unreliable on some systems. For production code, pre-convert PNGs to XPM using ImageMagick.

09 The Event Loop

`mlx_loop(mlx_ptr)`

```
mlx.mlx_loop(mlx_ptr)  # program runs here until exit
```

Starts the **infinite event loop**. This call **never returns** under normal operation. All logic now runs through callbacks. Must be called after all hooks are registered.

> ⚠ You cannot catch `Ctrl+C` inside `mlx_loop`. Use `Ctrl+\` to force-kill, or call `os._exit(0)` from a key hook.

`mlx_loop_exit(mlx_ptr)`

```
mlx.mlx_loop_exit(mlx_ptr)
```

Signals the loop to stop cleanly. `mlx_loop` will return, allowing code after it (cleanup) to run. Prefer this over `os._exit(0)` when you want to free resources properly.

10 Keyboard Hooks

`mlx_key_hook(win_ptr, callback, param)`

```python
def on_key(keycode, param):
    if keycode == 65307:  # ESC
        os._exit(0)

mlx.mlx_key_hook(win_ptr, on_key, None)
```

Registers a function called when a **key is released** (KeyRelease). Each window can have its own key hook.

`callback`	Function with signature `(keycode: int, param: any) → None`
`param`	Any Python object, passed back to the callback unchanged. Use to share state.

To **disable** the hook: `mlx.mlx_key_hook(win_ptr, None, None)`

COMMON KEY CODES (LINUX/XCB)

ESC 65307	Enter 65293	Space 32	Backspace 65288
Up ↑ 65362	Down ↓ 65364	Left ← 65361	Right → 65363
'a' 97	'z' 122	'1' 49	'2' 50
Delete 65535	Tab 65289	Shift L 65505	Ctrl L 65507

✓ Find any key code by printing it: `def on_key(key, p): print(key)`

11 Mouse Hooks

```
mlx_mouse_hook(win_ptr, callback, param)
```

```python
def on_mouse(button, x, y, param):
    if button == 1:  # left click
        print(f"Left click at ({x}, {y})")

mlx.mlx_mouse_hook(win_ptr, on_mouse, None)
```

Called when a **mouse button is pressed**. Provides the button code and the exact click coordinates within the window.

`button`	Which button: 1=Left, 2=Middle, 3=Right, 4=Scroll Up, 5=Scroll Down
`x, y`	Click position inside the window (top-left origin)

Clickable area hit-test:

```python
BTN_X, BTN_Y, BTN_W, BTN_H = 100, 200, 240, 55

def on_mouse(button, x, y, param):
    if button == 1:
        if BTN_X <= x <= BTN_X + BTN_W and BTN_Y <= y <= BTN_Y + BTN_H:
            do_action()
```

12 Expose Hook

> `mlx_expose_hook(win_ptr, callback, param)`

```
def on_expose(param):
    state.needs_redraw = True

mlx.mlx_expose_hook(win_ptr, on_expose, None)
```

Called when the window needs to be redrawn — e.g., after being uncovered or restored from alt-tab.

> ⚠ On modern composited desktops this event may fire only once at launch, or never. Don't rely on it alone. Use a `needs_redraw` dirty flag in your **loop hook** as your primary redraw mechanism.

13 Loop Hook

> `mlx_loop_hook(mlx_ptr, callback, param)`

```
def on_loop(param):
    if not state.needs_redraw:
        return                          # skip frame — nothing changed
    mlx.mlx_clear_window(mlx_ptr, win_ptr)
    draw_everything()
    state.needs_redraw = False

mlx.mlx_loop_hook(mlx_ptr, on_loop, None)
```

Called on every loop iteration when no other event is pending. This is the **primary place for rendering logic**. Bound to the `mlx_ptr` connection (not a specific window).

> ★ Use the **dirty flag pattern**: set `needs_redraw = True` only when state changes. Skip the redraw when nothing changed to avoid burning CPU on 60+ identical frames.

14 Generic Hook — mlx_hook

```
mlx_hook(win_ptr, x_event, x_mask, callback, param)
```

```
mlx.mlx_hook(win_ptr, 33, 0, on_close, None)  # window close button
```

The most powerful hook. Listens to **any X11 event**, beyond the three covered by convenience hooks.

X_EVENT	NAME	WHEN IT FIRES	CALLBACK SIGNATURE
2	KeyPress	Key held down (fires repeatedly if auto-repeat on)	(keycode, param)
3	KeyRelease	Key released (same as key_hook)	(keycode, param)
4	ButtonPress	Mouse button pressed	(button, x, y, param)
5	ButtonRelease	Mouse button released	(button, x, y, param)
6	MotionNotify	Mouse moved while a button is held	(x, y, param)
7	EnterNotify	Cursor enters the window	(param)
8	LeaveNotify	Cursor leaves the window	(param)
12	Expose	Window needs redraw	(param)
33	ClientMessage	Window close (X) button clicked	(param)

KeyPress (held down) vs KeyRelease:

```
# Fires immediately and repeatedly while key is held
mlx.mlx_hook(win_ptr, 2, 1, on_keypress, None)

# Fires once when key is released (standard mlx_key_hook behaviour)
mlx.mlx_hook(win_ptr, 3, 1, on_keyrelease, None)
```

Mouse enter / leave window:

```
mlx.mlx_hook(win_ptr, 7, 0, lambda p: setattr(state, 'mouse_in_win', True), None)
mlx.mlx_hook(win_ptr, 8, 0, lambda p: setattr(state, 'mouse_in_win', False), None)
```

ℹ **MotionNotify (event 6)** only fires while a mouse button is held. For hover detection without clicking, poll `mlx_mouse_get_pos` in your loop hook instead.

15 Mouse Utilities

`mlx_mouse_hide(mlx_ptr) / mlx_mouse_show(mlx_ptr)`

```
mlx.mlx_mouse_hide(mlx_ptr)   # cursor invisible
mlx.mlx_mouse_show(mlx_ptr)   # cursor visible again
```

Hides or shows the system mouse cursor. Use `mlx_mouse_hide` when drawing your own custom cursor image at the mouse position.

`mlx_mouse_move(mlx_ptr, x, y)`

```
mlx.mlx_mouse_move(mlx_ptr, 400, 300)
```

Moves the cursor programmatically to `(x, y)` inside the window.

`mlx_mouse_get_pos(mlx_ptr) → tuple`

```
ret, x, y = mlx.mlx_mouse_get_pos(mlx_ptr)
```

Returns the **current cursor position** without waiting for a click. Poll this in your loop hook for hover effects.

16 Keyboard Utilities

`mlx_do_key_autorepeatoff(mlx_ptr)`

```
mlx.mlx_do_key_autorepeatoff(mlx_ptr)  # one event per keypress
mlx.mlx_do_key_autorepeaton(mlx_ptr)   # back to default (rapid-fire while held)
```

By default, holding a key generates repeated events every second. `autorepeatoff` makes each physical press fire exactly once, useful when you want to detect a single press without handling duplicates.

17 Screen Info

`mlx_get_screen_size(mlx_ptr) → tuple`

```
ret, screen_w, screen_h = mlx.mlx_get_screen_size(mlx_ptr)
```

Returns the physical monitor resolution. Can be called before creating any window. Useful to decide window size, tile size, or to detect screen boundaries.

18 Sync & Flush

Controls when drawing commands are sent to the screen. Normally handled automatically by `mlx_loop` — you only need these for advanced frame control.

`mlx_do_sync(mlx_ptr)`

```
mlx.mlx_do_sync(mlx_ptr)  # flush all pending commands
```

`mlx_sync(mlx_ptr, cmd, img_or_win_ptr)`

```
mlx.mlx_sync(mlx_ptr, Mlx.SYNC_IMAGE_WRITABLE, img_ptr)  # wait before writing image
mlx.mlx_sync(mlx_ptr, Mlx.SYNC_WIN_FLUSH, win_ptr)      # force frame to appear
mlx.mlx_sync(mlx_ptr, Mlx.SYNC_WIN_COMPLETED, win_ptr)   # flush + wait for GPU done
```

CONSTANT	VALUE	WHAT IT DOES
`SYNC_IMAGE_WRITABLE`	1	Wait until the image buffer is safe to write again (GPU finished reading it)
`SYNC_WIN_FLUSH`	2	Send all pending draw commands for this window to the server
`SYNC_WIN_COMPLETED`	3	Flush AND wait for the server to confirm the frame is displayed

19 Color System

Colors are `unsigned int` values. The standard notation for `mlx_pixel_put` and `mlx_string_put` is `0xAARRGGBB`:

```
0xFF FF FF FF
  ↑  ↑  ↑  ↑
  A  R  G  B

Alpha:  0x00 = transparent,  0xFF = fully opaque
Note:   mlx_string_put forces alpha to 0xFF regardless
```

COMMON COLORS

COLOR	VALUE	COLOR	VALUE
White	0xFFFFFFFF	Black	0xFF000000
Red	0xFFFF0000	Green	0xFF00FF00
Blue	0xFF0000FF	Yellow	0xFFFFFF00
Cyan	0xFF00FFFF	Magenta	0xFFFF00FF
Grey	0xFF888888	Orange	0xFFFF8000

BUILD A COLOR FROM RGB COMPONENTS

```
def rgb(r, g, b, a=255):
    return (a << 24) | (r << 16) | (g << 8) | b

gold   = rgb(180, 140, 60)
purple = rgb(60, 30, 90)
```

For raw image buffer writes, byte order depends on the format returned by `mlx_get_data_addr`. On format 0 (B G R A): write bytes as `[blue, green, red, alpha]`.

20 Practical Recipes

A — Hover Effect (no click required)

```
def on_loop(param):
    _, mx, my = mlx.mlx_mouse_get_pos(mlx_ptr)
    hovering = (BTN_X <= mx <= BTN_X + BTN_W and
                BTN_Y <= my <= BTN_Y + BTN_H)
    img = img_hover if hovering else img_normal
    mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, img_bg, 0, 0)
    mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, img, BTN_X, BTN_Y)
```

B — Custom Mouse Cursor

```
mlx.mlx_mouse_hide(mlx_ptr)  # hide system cursor

def on_loop(param):
    _, mx, my = mlx.mlx_mouse_get_pos(mlx_ptr)
    mlx.mlx_clear_window(mlx_ptr, win_ptr)
    mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, img_bg, 0, 0)
    mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, img_cursor, mx - 8, my - 8)
```

C — Scene / State Machine (single window)

```
state.scene = "menu"

def on_loop(param):
    if not state.needs_redraw: return
    mlx.mlx_clear_window(mlx_ptr, win_ptr)
    if state.scene == "menu": draw_menu()
    elif state.scene == "game": draw_game()
    state.needs_redraw = False

def on_key(key, param):
    if state.scene == "menu" and key == 49:  # '1'
        state.scene = "game"
        state.needs_redraw = True
```

D — Key Held Down Detection

```
# event 2 = KeyPress: fires repeatedly while held
def on_keypress(key, param):
    if key == 65361:   # left arrow
        state.player_x -= 5
        state.needs_redraw = True

mlx.mlx_hook(win_ptr, 2, 1, on_keypress, None)
```

E — Multiple Windows

```
win1 = mlx.mlx_new_window(mlx_ptr, 800, 600, "Main")
win2 = mlx.mlx_new_window(mlx_ptr, 400, 300, "Secondary")

mlx.mlx_mouse_hook(win1, on_mouse_win1, None)
mlx.mlx_mouse_hook(win2, on_mouse_win2, None)

# Close only win2 without ending the program
mlx.mlx_hook(win2, 33, 0, lambda p: mlx.mlx_destroy_window(mlx_ptr, win2), Non
```

F — Clean Shutdown

```
def on_key(key, param):
    if key == 65307:  # ESC
        mlx.mlx_loop_exit(mlx_ptr)  # loop returns cleanly

mlx.mlx_loop(mlx_ptr)  # returns after mlx_loop_exit

# Cleanup runs here
mlx.mlx_destroy_image(mlx_ptr, img_ptr)
mlx.mlx_destroy_window(mlx_ptr, win_ptr)
mlx.mlx_release(mlx_ptr)
```

21 Limitations & Known Issues

No window positioning

Cannot set where on screen the window appears — the OS decides.

Font is fixed

`mlx_string_put` uses one built-in bitmap font. Size and face cannot be changed.

PNG loader unreliable

Some PNG types fail silently. Always convert PNGs to XPM with ImageMagick for production.

No Ctrl+C in loop

Python cannot catch SIGINT inside `mlx_loop`. Use `Ctrl+\` or `os._exit(0)` from a hook.

Expose event unreliable

May fire only once at launch on composited desktops. Use loop hook + dirty flag instead.

MotionNotify needs button

Event 6 only fires during click-drag. For hover without clicking, poll `mlx_mouse_get_pos`.

Text alpha forced to opaque

`mlx_string_put` always forces alpha to 255 — no semi-transparent text.

pixel_put is slow

Each call is a separate GPU command. Use image buffers for bulk pixel drawing.

No text input widget

No text field exists. Build your own using key hooks if needed.

Image format varies

`mlx_get_data_addr` returns format 0 or 1. Always check before writing raw pixel bytes.

22 Quick Reference Card

INIT

```
mlx_ptr = mlx.mlx_init()
mlx.mlx_release(mlx_ptr)
```

WINDOWS

```
win = mlx.mlx_new_window(mlx_ptr, w, h, "title")
mlx.mlx_clear_window(mlx_ptr, win)
mlx.mlx_destroy_window(mlx_ptr, win)
```

DRAW DIRECT (SLOW)

```
mlx.mlx_pixel_put(mlx_ptr, win, x, y, color)
mlx.mlx_string_put(mlx_ptr, win, x, y, color, "text")
```

IMAGES (FAST)

```
img = mlx.mlx_new_image(mlx_ptr, w, h)
data, bpp, sl, fmt = mlx.mlx_get_data_addr(img)
mlx.mlx_put_image_to_window(mlx_ptr, win, img, x, y)
mlx.mlx_destroy_image(mlx_ptr, img)
img, w, h = mlx.mlx_xpm_file_to_image(mlx_ptr, "f.xpm")  # recommended
img, w, h = mlx.mlx_png_file_to_image(mlx_ptr, "f.png")  # unreliable
```

HOOKS

```
mlx.mlx_loop(mlx_ptr)                                  # start — never returns
mlx.mlx_loop_exit(mlx_ptr)                              # exit from a hook
mlx.mlx_key_hook(win, lambda key,p: ..., None)         # key released
mlx.mlx_mouse_hook(win, lambda btn,x,y,p: ..., None) # click
mlx.mlx_expose_hook(win, lambda p: ..., None)          # window exposed
mlx.mlx_loop_hook(mlx_ptr, lambda p: ..., None)        # every frame
# generic:  2=KeyPress  3=KeyRelease  4=BtnPress  5=BtnRelease
#           6=Motion    7=Enter       8=Leave    33=WindowClose
mlx.mlx_hook(win, x_event, x_mask, fn, param)
```

MOUSE & KEYBOARD UTILS

```
mlx.mlx_mouse_hide(mlx_ptr)  /  mlx.mlx_mouse_show(mlx_ptr)
mlx.mlx_mouse_move(mlx_ptr, x, y)
ret, x, y = mlx.mlx_mouse_get_pos(mlx_ptr)
ret, w, h = mlx.mlx_get_screen_size(mlx_ptr)
mlx.mlx_do_key_autorepeatoff(mlx_ptr)  # one event per keypress
mlx.mlx_do_key_autorepeaton(mlx_ptr)   # default: rapid-fire while held
```

COLORS 0XAARRGGBB

```
0xFFFFFFFF White    0xFF000000 Black    0xFFFF0000 Red
0xFF00FF00 Green    0xFF0000FF Blue     0xFFFFFF00 Yellow
```

WRITTEN & COMPILED BY

Bruno Gomes

Based on MLX source by Olivier Crouzet — mlx_CLXV v2.2

PLATFORM

Linux — XCB / Vulkan — Python 3

Free to use, share, and improve